Rysunki, tabele i wykresy do rozdziałów 1, 3 i 4 (pkt 5)

Stan: 26.10.2026.

Gotowe rysunki leżą w `praca/rysunki/`. Każdy jest w PDF (do LaTeX-a, wektorowo) i PNG (podgląd). Diagramy mają też źródło `.puml` (PlantUML), żebyś mógł je poprawiać. Rysunki z danymi (para RGB–termowizja) pochodzą z Twojego zbioru – to dane własne, nie materiał wygenerowany.

Legenda statusu:

- ✓ gotowe – plik jest;
- ✍ do zrobienia przez Ciebie – zrzut ekranu, zdjęcie;
- 📝 tabela do wpisania – treść jest w `01_materialy_rozdzialy_1_3_4.md`.

Rozdział 1 – Wstęp

Nr	Propozycja	Status	Plik / skąd	Po

Rys. 1.1	Para obrazów tej samej zmiany: a) zdjęcie RGB ze znacznikiem Π, b) obraz termowizyjny (para 14)	✓	`para_14_rgb_termowizja.pdf`	Proble jedny rysunk znami widocz zdjęciu niewid na termog a znac widać Para 1 TRE ś mm, E folii 0,9 PanDe Dice 0
(opcja) Rys. 1.2	Schemat ideowy: wejście (2 obrazy) → system → wynik (raport z miarami)	–	można użyć uproszczonego `potok_przetwarzania` z rozdz. 3	Tylko Wstęp krótki. Zwykle wystar Rys. 1

- Inną parę wygenerujesz poleceniem `python praca\rysunki\para_rgb_termowizja.py 86` (Python z matplotlib i OpenCV, np. środowisko aplikacji).
- Wstęp zwykle nie ma tabel ani wykresów – nie dodawaj ich na siłę.

Podpis (propozycja): „Zmiana skórna z naklejonym znacznikiem Π: a) zdjęcie wykonane telefonem, b) obraz termowizyjny 320 × 240 px. Znamię, widoczne na zdjęciu między ramionami znacznika, nie jest widoczne na termogramie. Źródło: opracowanie własne.”

Rozdział 3 – Analiza projektowa

Nr	Propozycja	Status	Plik / skąd	
Tab. 3.1	Wymagania funkcjonalne WF1–WF14	📝	treść: `01_…`, sekcja 3.2	ko w pr
Tab. 3.2	Wymagania pozafunkcjonalne WN (kategorie, kryterium, sposób weryfikacji)	📝	treść: `01_…`, sekcja 3.3	w zr si (z „W ni
Rys. 3.1	Diagram przypadków użycia (2 aktorów, PU1–PU13, «include»/«extend»)	✓	`przypadki_uzycia.pdf` (`.puml`)	ab „E dz „U
Tab. 3.3	Scenariusz przypadku PU2 „Wykonaj rejestrację” albo PU8 „Wykonaj wszystkie etapy”	📝	szablon i scenariusze alternatywne: `01_…`, sekcja 3.4	1 w
Rys. 3.2	Architektura systemu (warstwy, moduły, dwa środowiska, dane)	✓	`architektura_systemu.pdf` (`.puml`)	po P os pr N O ap

Rys. 3.3	Potok przetwarzania (6 etapów, rozgałęzienie segmentacji, ocena CVAT warunkowa)	✓	`potok_przetwarzania.pdf` (`.puml`)	di ak U da ak po ro
(opcja) Rys. 3.4	Zdjęcie stanowiska: telefon, kamera termowizyjna, przedramię ze znacznikiem	✍	zrób zdjęcie przy kolejnej serii	po „c ni u 3.

Podpisy (propozycje):

- Rys. 3.1: „Diagram przypadków użycia systemu.”
- Rys. 3.2: „Architektura systemu: warstwy, moduły obliczeniowe i środowiska uruchomieniowe.”
- Rys. 3.3: „Potok przetwarzania pary obrazów (diagram aktywności UML).”

Rozdział 4 – Projekt systemu

Nr	Propozycja	Status	Plik / skąd	U
Tab. 4.1	Dobór technologii (element, technologia + wersja, uzasadnienie, alternatywa)	📝	`01_…`, sekcja 4.1 (wersje odczytane z Twoich środowisk)	–

Rys. 4.1	Rysunek wymiarowy znacznika Π: 35 × 26 mm, pasek 9 mm, otwór 17 × 17 mm = 289 mm², narożniki P1–P8	✓	`znacznik_pi_wymiary.pdf` (skrypt `znacznik_pi.py`)	numera narożni kolejnos `WZORZE` dopaso
Rys. 4.2	Znacznik na zdjęciu i na termogramie (wygląd folii w obu modalnościach)	✓	ten sam plik co Rys. 1.1 albo inna para	jeśli Rys jest we odwołaj niego z powtarz
Rys. 4.3	Diagram związków encji bazy SQLite	✓	`baza_erd.pdf` (zrobiony wcześniej)	–
Tab. 4.2	Opis tabel bazy (tabela, zawartość, klucze, relacje) + widoki	📝	`01_…`, sekcja 4.3	–
Rys. 4.4	Diagram klas (Prezentacja / Logika / Modele segmentacji; dziedziczenie, kompozycja, zależności)	✓	`diagram_klas.pdf` (`.puml`)	pokaza główne metody funkcyjn «utility»

Rys. 4.5	Zrzut ekranu aplikacji z opisanymi obszarami (1 – pasek etapów, 2 – panel sterowania, 3 – podglądy, 4 – status i log)	✍	zrób sam (instrukcja niżej)	kluczow do 4.5
(opcja) Rys. 4.6	Zrzut zakładki „Pomiary” albo „Raport” z podglądem E2	✍	jw.	może tr Dodatku (instruk

Jak zrobić Rys. 4.5:

1. Uruchom aplikację.
2. Wczytaj parę 14 i kliknij „Wykonaj wszystkie etapy”.
3. Przejdź do zakładki „2. Rejestracja” (nałożenie z suwakiem) albo „3. Segmentacja”.
4. Zrób zrzut okna: `Win + Shift + S`, wybierz okno.
5. Numery obszarów (1–4) dodaj w dowolnym edytorze albo w LaTeX-u (TikZ na obrazie) – wolisz drugie, bo tekst pozostaje edytowalny.

Podpisy (propozycje):

- Rys. 4.1: „Wymiary znacznika referencyjnego Π; P1–P8 – narożniki wykorzystywane do wyznaczenia skali i błędu TRE.”
- Rys. 4.4: „Diagram klas systemu (z pominięciem metod pomocniczych).”
- Rys. 4.5: „Okno aplikacji – zakładka rejestracji: 1 – pasek etapów, 2 – panel sterowania, 3 – podgląd nałożenia, 4 – status i dziennik zdarzeń.”

Wykresy

W rozdz. 1, 3, 4 wykresy zwykle nie są potrzebne – wyniki (histogramy Dice, TRE, porównanie E4) należą do rozdz. 6 i są w raporcie zbiorczym.

Jedyny sensowny wykres tutaj, opcjonalny: **czasy etapów** (rejestracja, segmentacja M1, M3, raport) jako uzasadnienie wymagania wydajnościowego WN. Wymaga pomiaru na Twoim komputerze. Mój pomiar w maszynie wirtualnej (2 rdzenie CPU, 5 par):

- rejestracja 5,3–8,0 s, średnio 6,5 s;
- U-Net ok. 67 ms na obraz;
- PanDerm nie był mierzony (wymaga GPU).

Lepiej podać te liczby w tabeli WN niż na wykresie.

Wstawianie w LaTeX-u

```
\begin{figure}[htbp]
  \centering
  \includegraphics[width=\textwidth]{rysunki/przypadki_uz
  \caption{Diagram przypadków użycia systemu.}
  \label{fig:przypadki-uzycia}
\end{figure}
```

Szerokości (orientacyjnie):

- `architektura_systemu`, `diagram_klas`: `\textwidth`;
- `potok_przetwarzania`: `0.75\textwidth`;
- `znacznik_pi_wymiary`: `0.6\textwidth`;
- `para_14_rgb_termowizja`: `\textwidth`.

Edycja diagramów (.puml)

Najprościej w VS Code:

1. Zainstaluj rozszerzenie „PlantUML” (jebbs). Wymaga Javy, a do diagramów klas i przypadków użycia – Graphviz.
2. Otwórz plik `.puml`.
3. Eksportuj przez „PlantUML: Export Current Diagram” → SVG/PNG.

PDF z SVG zrobisz np. w Inkscape. Wspólny styl (czarno-biały, bez cieni) jest w `styl.iuml`. Diagramy powstały w PlantUML 1.2019.06; nowsze wersje mogą nieco inaczej rozmieścić elementy.